

문제지는 양면으로 5쪽(pages)입니다, 문제지를 분리하지 마세요.
박스 내부 또는 문제 사이의 여백 분량이 답안을 작성할 수 있는 최대 분량입니다.
모든 문제의 풀이 과정은 해답지에 작성하세요.

- 1. 레지스터 중에서 MAR과 MBR의 기능을 설명하시오.
- 2. 컴퓨터에서 **파이프라인(Pipeline)** 기술의 개념을 설명하고, 파이프라인이 CPU 성능 향상에 어떻게 기여하는지 설명하시오.
- 3. 폰 노이만의 내장형 프로그램 방식에서의 명령어 수행 사이클의 3단계를 나누어 설명하시오. ‘PC 레지스터’, ‘IR 레지스터’ 이 2개의 단어를 필수적으로 사용해야합니다.
- 4. 캐시 메모리는 associative memory 형식이라고 한다. 이 의미를 설명하시오.
- 5. 캐시 메모리의 hit ratio란 무엇인가? 이것이 왜 중요한가?
- 6. 아래의 C 코드와 동일한 기능을 수행하는 Pep/8 기계어 코드이다. 아래의 음영으로 표시된 각 셀에 하나씩의 Pep/8 기계어 명령어를 **16진수**로 기입하시오.

#include <stdio.h>	49	00	20
char ch1, ch2;	49	00	21
int main() {			
scanf(“%c%c”, &ch1, &ch2);			
printf(“%c%c”, ch1, ch2);			
return 0;	00		
}	zz		

7. 아래의 Pep/8 기계어 코드가 수행을 완료하면, 주기억장치 0x20 번지와 0x21 번지 각각의 숫자를 16 진수로 쓰고, 이 2 바이트를 2 의보수로 해석했을때의 숫자를 10 진수로 적으시오.

```
11000000 00000000 00000100
10000000 00000000 00000110
11100001 00000000 00100000
00000000
```

8. [심화 문제]아래의 Pep/8 기계어 코드를 수행했을때의 출력값을 쓰시오. 코드 해독 과정을 각 코드 줄의 우측에 적으시오.

```
D1 00 1D
F1 00 09
C1 00 17
71 00 19
A1 00 1B
F1 00 16
51 00 16
00
00
00 05
00 03
00 30
81
zz
```

9. [심화 문제] 아래의 C 코드를 Pep/8 어셈블리어로 변환하려고 한다. 아래의 if 문을 변환할 아이디어를 제시하시오. 수업 중에 사용하지 않은 인스트럭션을 사용할 수도 있습니다.

```
int number = 3;
if (number % 2 == 0)
```

10. 아래의 C 코드와 동일한 기능을 수행하는 Pep/8 어셈블리어 코드이다. 빈 곳을 모두 채우시오.

<pre>#include <stdio.h> int num; int sum; int main() { printf("enter a number"); scanf("%d", &num); sum = num + 10; printf("%d", sum); return 0; }</pre>	<pre>BR main num: .block 2 sum: .block 2 bonus: msg: .ascii "enter a number: \x00" main: STRO msg, d DECI num, d LDA num, d ADDA bonus, i STOP .END</pre>
---	---

11. 아래의 C 코드와 동일한 기능을 수행하는 Pep/8 어셈블리어 코드이다. 빈 곳을 모두 채우시오

<pre>#include <stdio.h> int input; int sum = 0; int main() { printf("enter numbers(0: quit): \n"); while (true) { scanf("%d", &input); if (input == 0) { break; } sum += input; } printf("sum: %d\n", sum); return 0; }</pre>	<pre>BR main input: .block 2 sum: .block 2 msg1: .ASCII "enter numbers(0: quit): \x00" msg2: .ASCII "sum: \x00" main: STRO msg1, d while: DECI input, d CPA 0, i ADDA sum, d STA sum, d endWhile:STRO msg2, d DECO sum, d STOP .END</pre>
--	--

12. 아래의 C 코드의 Pep/8 어셈블리어 변환 결과이다. 빈 곳을 메우시오.

```
#include <stdio.h>
```

```
char ch;
```

```
int main(){
```

```
    scanf("%c", &ch);
```

```
    if ( ch >= 'A' && ch <= 'Z')
```

```
        printf("Capital Letter");
```

```
    else
```

```
        print("Not Capital Letter");
```

```
    return 0;
```

```
}
```

```
BR    main
```

```
main:  CHARI ch, d
```

```
if:    LDA 0x0000, i
```

```
        LDBYTEA ch, d
```

```
        CPA 'A', i
```

```
        BRLT else
```

```
BR endIf
```

```
else:  STRO msg2, d
```

```
endIf: STOP
```

```
msg1:  .ASCII "Capital Letter\x00"
```

```
msg2:  .ASCII "Not Capital Letter\x00"
```

```
.END
```

Instruction Specifier	Mnemonic	Operation	Addressing Modes	Status Bits
0000 0000	STOP	Stop execution	U	
0000 0001	RETTR	Return from trap	U	
0000 0010	MOVSPA	Move SP to A	U	
0000 0011	MOVFLGA	Move NZVC flags to A	U	
0000 010a	BR	Branch unconditional	i, x	
0000 011a	BRLE	Branch if less than or equal to	i, x	
0000 100a	BRLT	Branch if less than	i, x	
0000 101a	BREQ	Branch if equal to	i, x	
0000 110a	BRNE	Branch if not equal to	i, x	
0000 111a	BRGE	Branch if greater than or equal to	i, x	
0001 000a	BRGT	Branch if greater than	i, x	
0001 001a	BRV	Branch if V	i, x	
0001 010a	BRC	Branch if C	i, x	
0001 011a	CALL	Call subroutine	i, x	
0001 100r	NOTr	Bitwise invert r	U	NZ
0001 101r	NEGr	Negate r	U	NZV
0001 110r	ASLr	Arithmetic shift left r	U	NZVC
0001 111r	ASRr	Arithmetic shift right r	U	NZC
0010 000r	ROLr	Rotate left r	U	C
0010 001r	RORr	Rotate right r	U	C
0010 01nn	NOPn	Unary no operation trap	U	
0010 1aaa	NOP	Nonunary no operation trap	i	
0011 0aaa	DECI	Decimal input trap	d, n, s, sf, x, sx, sxf	NZV
0011 1aaa	DECO	Decimal output trap	i, d, n, s, sf, x, sx, sxf	
0100 0aaa	STRO	String output trap	d, n, sf	
0100 1aaa	CHARI	Character input	d, n, s, sf, x, sx, sxf	
0101 0aaa	CHARO	Character output	i, d, n, s, sf, x, sx, sxf	
0101 1nnn	RETn	Return from call with n local bytes	i, d, n, s, sf, x, sx, sxf	
0110 0aaa	ADDSP	Add to stack pointer (SP)	i, d, n, s, sf, x, sx, sxf	NZVC
0110 1aaa	SUBSP	Subtract from stack pointer (SP)	i, d, n, s, sf, x, sx, sxf	NZVC
0111 raaa	ADDr	Add to r	i, d, n, s, sf, x, sx, sxf	NZVC
1000 raaa	SUBr	Subtract from r	i, d, n, s, sf, x, sx, sxf	NZVC
1001 raaa	ANDr	Bitwise AND to r	i, d, n, s, sf, x, sx, sxf	NZ
1010 raaa	ORr	Bitwise OR to r	i, d, n, s, sf, x, sx, sxf	NZ
1011 raaa	CPr	Compare r	i, d, n, s, sf, x, sx, sxf	NZVC
1100 raaa	LDr	Load r from memory	i, d, n, s, sf, x, sx, sxf	NZ
1101 raaa	LDBYTer	Load byte from memory	i, d, n, s, sf, x, sx, sxf	NZ
1110 raaa	STr	Store r to memory	d, n, s, sf, x, sx, sxf	
1111 raaa	STBYTer	Store byte r to memory	d, n, s, sf, x, sx, sxf	